

Часть 4. Burst: магия нативного кода в C#

В прошлой части — Jobs и параллелизм: `IJobEntity`, `ScheduleParallel`, race conditions, `ComponentLookup`. ECS-симуляция через Job System разносит работу на worker-потoki, но **реальное ускорение приходит когда внутри job'ов работает Burst**.

Burst — это AOT-компилятор Unity, который превращает managed C# в нативный машинный код с SIMD-инструкциями. Один атрибут `[BurstCompile]` — и `EnemyMoveJob` **ускоряется в x33** (с реальными замерами ниже).

Разбор паттерна на примере Survival-фичи: top-down arena с волнами врагов и автоматической стрельбой.

Что такое Burst

Обычный C# в Unity выполняется через Mono runtime. JIT-компилятор переводит IL → нативный код **на лету** при первом вызове метода. Это компромисс: универсально для любых платформ, но медленно и аллокирует managed-объекты.

Burst — это **отдельный pipeline**. Он берёт ограниченное подмножество C# (без managed-типов, без reflection, без managed exceptions), компилирует **заранее** через LLVM в нативный код, агрессивно применяет vectorization и использует SIMD-инструкции (AVX2 на десктопе, NEON на ARM-устройствах).

Один атрибут — `[BurstCompile]` — превращает обычный job в нативно-исполняемый код:

```
[BurstCompile]
[WithAll(typeof(EnemyTag))]
public partial struct EnemyMoveJob : IJobEntity
{
    public float3 PlayerPos;
    public float DeltaTime;

    private void Execute(ref LocalTransform transform,
                        in MoveSpeed speed,
                        in RotationSpeed rotationSpeed,
                        in ContactDamage cd)
    {
        // math.distance, math.normalize, math.slerp ...
    }
}
```

Без `[BurstCompile]` — это исходник выполняется через Mono runtime, со всем сопутствующим overhead'ом. С `[BurstCompile]` — это **тот же исходник**, но физически другой бинарь: LLVM-скомпилированный SIMD-код.

Атрибут также применяется к системам:

```
[BurstCompile]
public partial struct EnemyMoveToPlayerSystem : ISystem
{
    [BurstCompile]
    public void OnUpdate(ref SystemState state) { /* ... */ }
}
```

Что внутри Burst разрешено

Burst — **строгий**. Не весь C# можно использовать внутри `[BurstCompile]`-метода:

✅ Можно	❌ Нельзя
<code>Unity.Mathematics</code> (<code>math.distance</code> , <code>quaternion.slerp</code> и т.д.)	<code>UnityEngine.*</code> (<code>Vector3</code> , <code>Quaternion</code> , <code>Mathf</code>)
<code>NativeArray</code> , <code>NativeList</code> , <code>NativeHashMap</code>	<code>List<T></code> , <code>Dictionary<T></code> , managed-массивы
<code>Entity</code> , <code>ComponentLookup<T></code> , <code>EntityCommandBuffer</code>	Managed-объекты (<code>GameObject</code> , <code>Transform</code> , <code>Animator</code>)
Простой контрол-флоу: <code>if</code> , <code>switch</code> , <code>for</code> , <code>while</code>	<code>Debug.Log</code> (managed string)
<code>unsafe</code> указатели, ptr-арифметика	LINQ, reflection, dynamic
<code>static readonly</code> поля примитивных типов	Создание <code>new</code> managed-объектов в hot path

При нарушении правил — **compile error**. Burst-компилятор громко скажет «эту строку я не умею». Хорошая новость: проблемы вылавливаются на этапе компиляции, не в runtime.

Внутри Survival-фичи это форсирует **disciplined data-only code**: все компоненты — простые `struct`, все вычисления — через `Unity.Mathematics`, никаких managed-вызовов внутри job'а. Что, собственно, и нужно для cache-friendly performance.

Замер на живом проекте

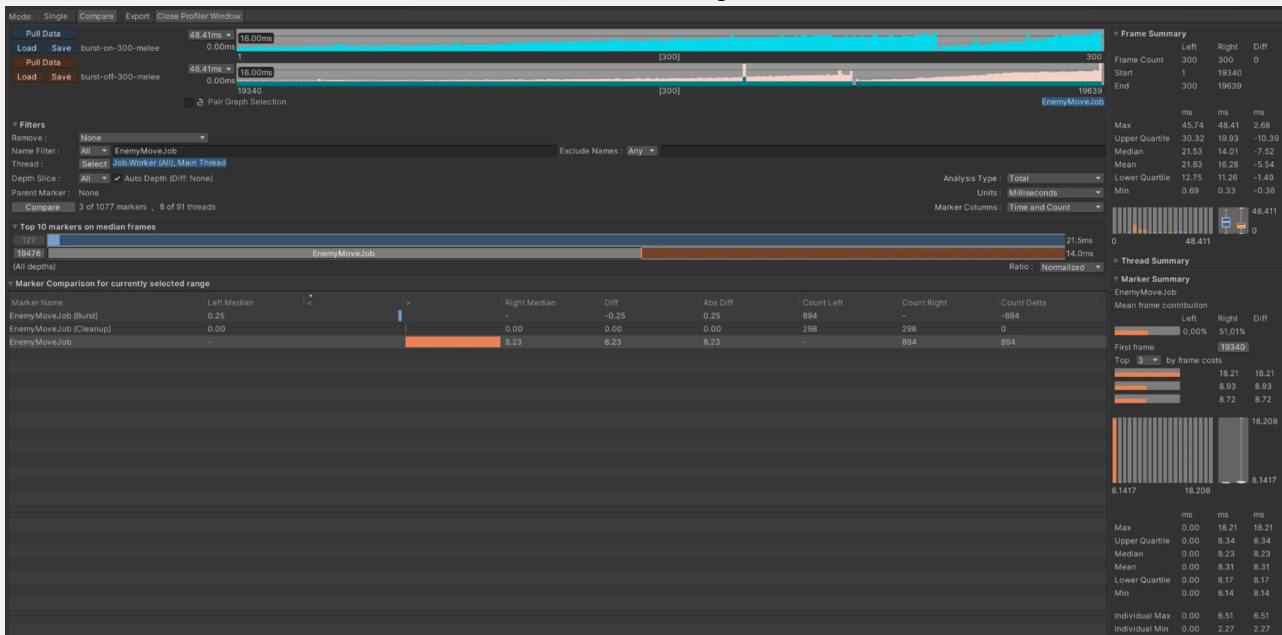
Чтобы цифры были не из tutorial-вакуума, ниже — замер из Survival-фичи через **Profile Analyzer**.

Сценарий:

- 300 ECS-entities (cube-врагов), spawned одним initial burst'ом
- Все сбились в melee-кластер вокруг игрока — то есть separation loop в `EnemyMoveJob` активно работает на $O(n^2)$
- Editor mode, Unity 2022.3.51f1
- VSync = Don't Sync, `Application.targetFrameRate = -1`

Метрика — `EnemyMoveJob` (главный «тяжёлый» job в проекте, параллельный):

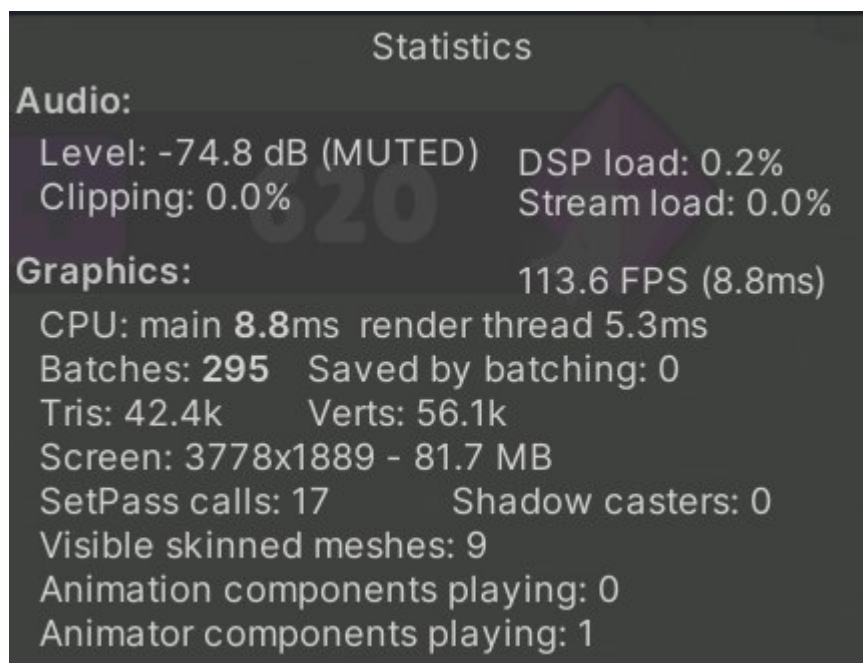
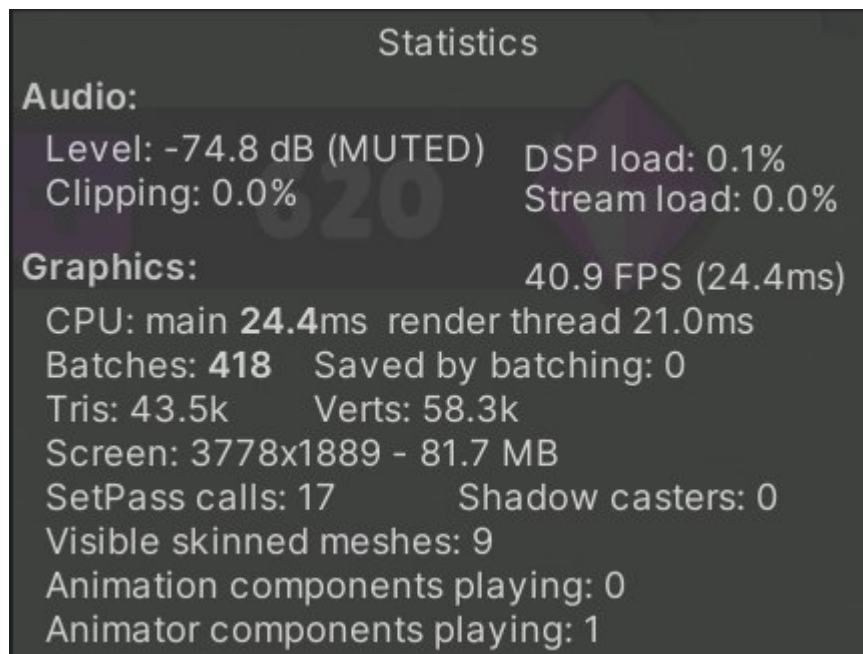
 Screenshot 1: Profile Analyzer Compare mode, Name Filter = *EnemyMoveJob*. Видны две строки — *EnemyMoveJob (Burst)* Left=0.25ms и *EnemyMoveJob* Right=8.23ms.



	Burst ON	Burst OFF	Speedup
-----	-----	-----	-----
EnemyMoveJob median (ms)	0.25	8.23	x33
EnemyMoveJob max (ms)	0.36	18.21	x51
EnemyMoveJob range (ms)	0.14	10.07	x72 хуже
% of frame consumed	~1%	51.01%	—

Frame-level (Unity Stats overlay):

	Burst ON	Burst OFF	Speedup
-----	-----	-----	-----
FPS	113.6	40.9	x2.78
Frame time (ms)	8.8	24.4	x2.77
CPU main (ms)	8.8	24.4	x2.77
Render thread	5.3	21.0	—



 Screenshot 2: Stats overlay npu Burst ON (114 FPS / 8.8ms) рядом с Stats overlay npu Burst OFF (41 FPS / 24.4ms).

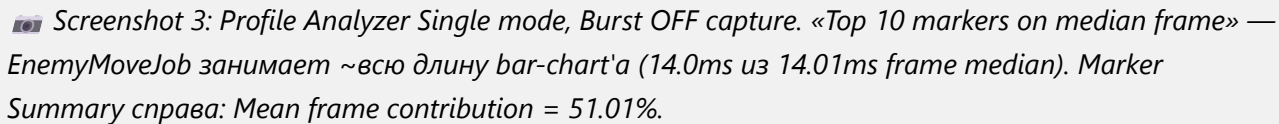
Три инсайта из этих цифр

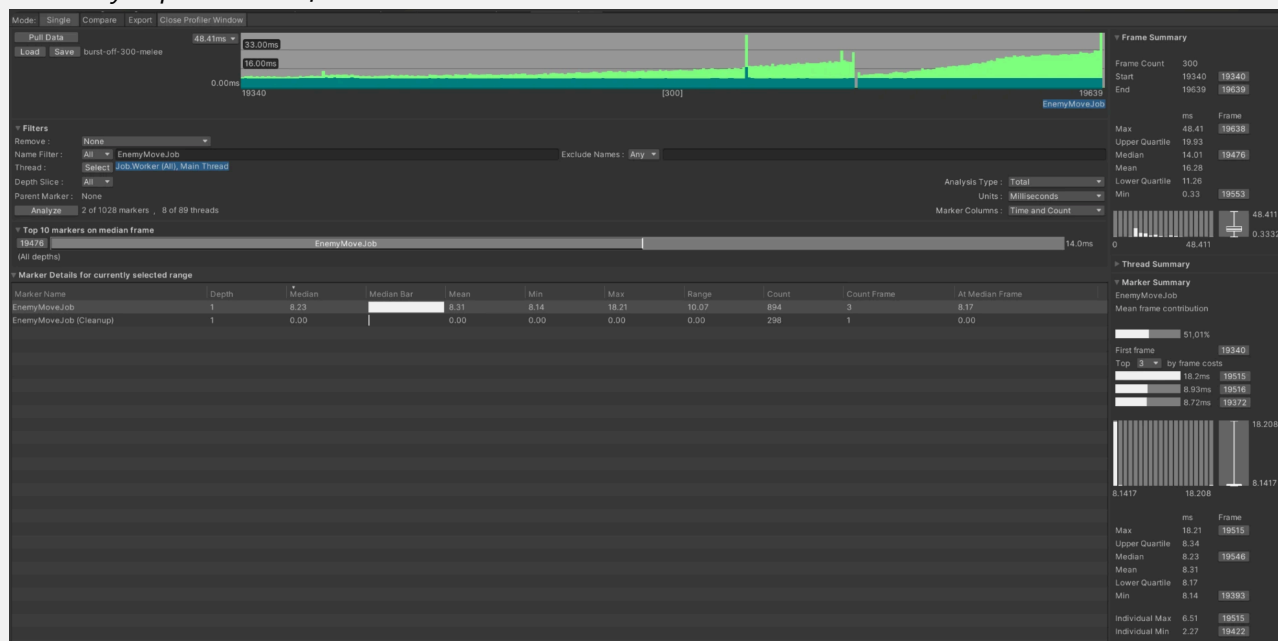
1. Burst поднимает FPS почти в ×3

С 41 FPS (неиграбельно) → 114 FPS (комфортно) — на тех же 300 entities. **Один атрибут**, один restart Play mode.

Это **наивный** FPS-замер. Реальная история в job-level: ×33 на EnemyMoveJob. Frame-level «всего» ×2.78 — потому что рендеринг 300 cube'ов, UI-канвас, animator на игроке и Editor-overhead тоже едят кадр **независимо от Burst**.

2. Без Burst один job съедает половину кадра

 Screenshot 3: Profile Analyzer Single mode, Burst OFF capture. «Top 10 markers on median frame» — EnemyMoveJob занимает ~всю длину bar-chart'a (14.0ms из 14.01ms frame median). Marker Summary справа: Mean frame contribution = 51.01%.



51% frame budget — один job на 300 cube'ax. С Burst тот же job становится **noise** в profile — теряется среди других маркеров, занимая ~1% кадра.

В реальной production-игре это значит: **те же CPU-cycles освобождаются под другую логику** — физику, AI, рендеринг, gameplay-системы. Burst — это не только «×3 FPS», это «больше пространства для остального».

3. Burst даёт **stability**, а не только speed

Range (разброс между min и max) на одном и том же job:

- **Burst ON:** 0.14ms (от 0.21 до 0.36) — job-время **почти константа**
- **Burst OFF:** 10.07ms (от 8.14 до 18.21) — job-время **скачет в 2.2 раза от кадра к кадру**

Это GC pauses, JIT-discrepancies, branch mispredictions, всё что характерно для managed runtime. Burst компилирует код заранее в нативный — нет JIT, нет managed-аллокаций, нет GC pressure → стабильные frame times.

30 FPS стабильно лучше 60 FPS с фризами на 100ms. Stability часто важнее raw speed — особенно для VR / mobile, где micro-jitter ощущается физически.

Burst Inspector

Jobs → Burst → Open Inspector — отдельное окно, которое показывает **сгенерированный assembly** для каждого [BurstCompile]-метода.

Левая панель — список всех Burst-jobs в проекте. Правая — реальный x64 / ARM64 код с SIMD-инструкциями:

```
vmulps      ymm1, ymm0, [rdi + 0x10]
vfmadd231ps ymm2, ymm1, [rdi + 0x20]
vbroadcastss ymm3, dword ptr [rdi + 0x30]
```

`vmulps`, `vfmadd231ps`, `vbroadcastss` — это **AVX2-инструкции**, которые обрабатывают **8 float'ов за один такт CPU**. Compiler нашёл циклы по компонентам и автоматически векторизовал.

Это не магия — это LLVM, который видит data flow целиком. Никакой reflection, никакой virtual dispatch, никаких managed string-операций — компилятор оптимизирует **агрессивно**, потому что в job'e работают только `Unity.Mathematics` типы фиксированной формы.

Mobile: AOT обязателен

На iOS **JIT-компиляция запрещена** на уровне ОС (всё кроме system-level code должно быть подписано). Это значит:

- **Без Burst**: на iOS все ECS-jobs пойдут через **Mono interpreter** — даже медленнее чем Mono JIT на десктопе.
- **C Burst (AOT)**: код скомпилирован **заранее**, упакован в IPA, исполняется как нативный ARM64.

На Android Mono JIT работает, но всё равно медленнее AOT.

Burst — это не optional оптимизация для mobile-таргета. Без него ECS на мобильных платформах теряет главное преимущество — нативный perf.

В `Edit` → `Preferences` → `Burst AOT Settings` проверь:

- **Enable Burst Compilation** = ON для нужных target-платформ
- **Use Platform SDK Linkers** = ON (для iOS device build)

Trade-offs

Burst не бесплатен:

Цена	Импакт
Долгий первый build	+20–60 секунд на компиляцию Burst-кода при первом запуске Play / Build
Сложная отладка	Breakpoint'ы в Burst-jobs не работают как в managed; отлаживай через Burst Inspector или временно отключай Burst для конкретного job'a
Compile-time errors при нарушении правил	Каждый раз при написании job'a — приходится знать что Burst-friendly, что нет
Не работает с managed UnityEngine API	Полный data-only код в hot path: нет <code>GetComponent</code> , <code>Instantiate</code> , <code>transform.position</code>

Эти ограничения — **не баги**, а закономерное следствие AOT-компиляции и data-oriented дизайна. Burst — это договор: «ты пишешь disciplined data-only код, я даю тебе нативный perf».

Что дальше

На этом серия по ECS DOTS заканчивается:

- **Часть 1** — почему ECS быстрее (AoS vs SoA, GC, virtual dispatch)
- **Часть 2** — из чего состоит ECS (Entity, Components, Systems, Baker)
- **Часть 3** — параллелизм без race conditions (Jobs)
- **Часть 4** — нативный код через Burst (*вы здесь*)

Стек устоялся: **ECS + Jobs + Burst** — это production-ready инструмент для случаев где нужны сотни-тысячи однотипных entities с жёстким perf-бюджетом. Не silver bullet, но и не экспериментальный prototype — реальный pipeline, на котором уже сегодня делают игры в индустрии.

Связанная литература

- [Unity Burst Compiler documentation](#)
- [Burst Inspector overview](#)
- [Unity Mathematics package](#)
- Часть 3 серии — «Jobs: параллелизм без race conditions»